

Parts Library and Criteria Format

The minimum set of reusable functional parts, the acceptance-criteria format that keeps their checks composable when assembled into one application, the composition rules, and what remains unsolved — with real format examples and a two-part worked proposal.

1. The recommended parts list

1.1 What the census found

Fourteen enumerable sources were cross-tabulated: internal-tool platforms (Retool's ~127 components, OutSystems Forge with its download counts, Appsmith, Budibase, NocoBase, Softr, Mendix), SaaS starter kits (ShipFast, SaaS Pegasus, Laravel Jetstream, Vercel's Next.js SaaS starter, Wasp/Open SaaS, create-t3-app), and Supabase as the platform-as-parts-bin. The WordPress popular-plugins list — the largest natural experiment in what people bolt onto applications, with public active-install counts — corroborates the frequencies: Contact Form 7 at 10M+ active installs, WooCommerce 7M+, WPForms 5M+, WP Mail SMTP 4M+ (wordpress.org).

Two blocks appear essentially everywhere: **authentication** (11 of 14 sources; all seven starters ship the identical login/registration/session/2FA quartet — Jetstream's own description is "login, registration, email verification, two-factor authentication, session management" ([docs](#))) and **forms** (13 of 14). One block is an entire product category: **CRUD tables over a database with an admin surface** is what Retool, Appsmith, Budibase and NocoBase fundamentally are. The rest of the majority tier: **roles/permissions** (8), **payments/billing** (7 — every customer-facing starter, near-zero in internal-tool templates), **transactional email/notifications** (8), **file upload/storage** (7), **charts/reporting** (7), and **workflow/approvals/background jobs** (7 — with the striking datum that 22 of Budibase's 23 templates are approval flows ([templates](#))).

1.2 The recommended list

Core nine — in, because they recur across effectively every source:

1. **Auth** (login, registration, sessions, password reset, optional SSO/2FA)
2. **Forms** (data collection with validation)
3. **Records** (CRUD over a datastore, list/detail/edit, the admin panel case included)
4. **Permissions** (roles, per-role read/write/action gates)
5. **Notify** (transactional email, in-app, optional SMS/push)
6. **Files** (upload, storage, retrieval, presigned access)
7. **Reports** (charts/dashboards over the records)
8. **Flow** (approvals, background jobs, scheduled tasks)
9. **Billing** (checkout, subscriptions, webhooks) — *conditional: present in every customer-facing source, absent from internal tools; ship it as a first-class part but expect roughly half of assembled apps to exclude it*

Second shelf — in the library but not the core, because they appear in a minority of sources and each is a whole service category when done properly: teams/multi-tenancy; search (long tail as a shipped block — 2 of 14 — though universal as a bolt-on service); calendar/scheduling (note Cal.com is an entire company shipping this one part); audit log; content/SEO pages; AI chat (5 of 14 and rising).

Out, with the reasoning: device capabilities, maps, kanban, PDF generation, i18n, feature flags, impersonation — each appears in one to three sources. They are real demands (OutSystems' Ultimate PDF has 25,096 downloads ([Forge](#))) but each added part carries the full cost of criteria authoring, configuration schema and composition testing while covering a shrinking slice of apps.

1.3 Where the set stops, and what is always custom

The stopping rule the evidence supports has two triggers, and the vendors state them themselves. OutSystems names the moment to leave the blocks: when you need "your own algorithms to solve your specific use cases" ([custom-code post](#)) — its platform "promotes the graceful 'stepdown' to standard 3GL languages" at exactly that point ([evaluation guide](#)). Bubble's documented hard limits give the second trigger: 300-second workflow timeouts, 50,000-record sorted searches, 10,000 elements per page ([hard limits](#))

— when the unit of work no longer fits one block's envelope, composition stops and custom build starts.

Always business-specific, in the vendors' own words: ShipFast covers the integrations so you can "spend your time building your startup" — the startup is not in the box (shipfast.com); Pegasus "handles the foundation of your application" — foundation, not application (aaspegasus.com). The never-templated residue is consistent across all sources: **the domain data model, the proprietary algorithm (pricing, matching, scoring — the thing the business actually is), and org-specific approval semantics** — even Budibase, whose entire catalogue is approvals, ships them as thin starting points because every company's process differs.

Verdict: the parts list is the easy half. Nine core parts plus a conditional Billing cover the recurring functional surface of business applications; the census sources agree with each other to a degree that suggests this is settled industry knowledge. The set stops at proprietary logic and at envelope limits, and both boundaries are documented by the platforms themselves.

2. The criteria format

2.1 What exists, and the zero-interpretation test

Every shipping format was judged against one question: given only the artifact, can a machine produce pass/fail with zero human or model interpretation?

Gherkin fails it, and this needs saying plainly because it is the industry's default answer. A `.feature` file is not executable. Cucumber's own docs: "When Cucumber executes a Gherkin step in a scenario, it will look for a matching step definition to execute" ([step definitions](#)) — the meaning of every step lives in glue code someone writes per application. Gauge and Robot Framework (for app-specific keywords) share the disease.

The formats that pass are the ones with a *closed* vocabulary — every keyword defined by the runtime, nothing left to a glue author. Hurl is the cleanest existence proof ([asserting-response](#)):

```
GET https://example.org/api/cats
HTTP 200
[Asserts]
jsonpath "$.cats" count == 49
duration < 1000
```

Karate passes for its HTTP surface (closed keyword set, first-class retry: `* configure retry = { count: 10, interval: 2000 }` then `retry until response.id == 1` ([polling docs](#))). StepCI and Maestro pass. Playwright passes *once written* but is code — authoring is the interpretation step. Pact's contract JSON passes for replay, but its provider states are named preconditions with hand-written handlers. Schemathesis/Microcks pass but express only *shape* criteria — they cannot say "the order was actually created."

The newest and most relevant standard is Arazzo (OpenAPI Workflows, v1.1.0 published 17 May 2026 — a real spec, early adoption, runner ecosystem still thin ([spec](#); [APIScout survey](#))). Its Criterion Object is the best existing skeleton:

```
successCriteria:
  - context: $response.body
    condition: $[?count(@.pets) > 0]
    type: jsonpath
```

— *what to look at* (`context`), *the decision rule* (`condition`), *the rule's language* (`type`), plus step-level `timeout` , `retryAfter` , `retryLimit` , and namespaced outputs (`$steps.loginStep.outputs.sessionToken`). But it deliberately stops at the API surface: no UI channel, no invisible-outcome channel.

2.2 Your four-field hypothesis, corrected

You proposed trigger + observable outcome + where observed + decision rule. The formats' own required fields say that is **necessary but not sufficient** — **two fields are missing and one needs splitting**:

1. **Precondition / starting state** — every composable format forces it: Gherkin `Given` , Pact `given('I have a list of dogs')` , Arazzo `dependsOn` + `inputs` , Playwright project `dependencies` + `storageState` . Without it a criterion is only unambiguous relative to unstated world-state.
2. **Trigger** — confirmed.

3. **Observation channel + address** — your "where," made explicit: the formats force you to name the *channel* (HTTP response, DOM, span, mailbox), not just a location.
4. **Extraction query** — split out from "where" by every mechanical format (JSONPath, XPath, span attribute, header name).
5. **Decision rule from a closed matcher set** — confirmed, with "closed set" as the load-bearing property.
6. **Time budget** — *missing from your list, and every serious format converged on forcing it*: Playwright auto-retries assertions for 5 seconds by default; Maestro waits up to 7; Arazzo has `timeout / retryAfter / retryLimit`; Hurl has `--retry`. Without a time budget, "the email is sent" is undecidable — there is no moment at which failure is declared.

2.3 Invisible outcomes — your suspicion confirmed

This is the weakest area in existing work, exactly as you expected. No declarative format has first-class email, webhook, queue or database-row outcome channels. In practice these are checked by *interposing infrastructure* driven by imperative code: email goes through a capture service and is asserted via its API (Mailosaur:

`messages.get(serverId, { sentTo: ... })`), which "automatically waits for the first email to arrive that matches" ([docs](#)); self-hosted Mailpit ships "a REST API for integration testing" ([repo](#)); webhooks through an interposed receiver (`stripe listen --forward-to ...` — with Stripe's own warning that event *ordering* is not guaranteed and therefore not assertable ([docs](#)); queues and rows by polling a database or admin API (Karate's JDBC helper plus `retry until`).

The single structural exception: **Tracetest** reduces every *instrumented* internal effect to one declarative channel — OpenTelemetry spans — with selectors and polling profiles:

`selector: span[name = "POST /pokemon/import"] / attr:http.status_code = 200` ([docs](#)). The price: the app must be instrumented, and your telemetry schema becomes your criterion vocabulary. Its OSS maintenance is a watch item.

The design consequence for your library: the observation channel cannot be an afterthought of the criteria format — it must be part of the *part's own definition*. A part that sends email must ship, as part of its contract, the requirement that the assembled app expose a mail-capture endpoint in test tenancy. The part declares the channel; the assembly provides it; the criterion addresses it.

2.4 The proposed format

Fields, then two worked examples. This is a *proposal* assembled from shipping pieces (Arazzo's criterion object, Pact's named states, Tracetest's channel concept, Playwright's auto-wait defaults) — labelled as such.

Each part ships a `criteria.yaml` containing: `part` (name + version), `config_schema` (the questions this part needs answered, JSON-Schema typed), `provides[]` (named state capabilities this part can establish, with outputs — e.g. `authenticated_session` yielding `session_token`), `requires[]` (capabilities that must be provided by other parts or the assembly, including observation channels), `controls[]` (every control: role + accessible name template, endpoint, states — your existing surface record), and `criteria[]`. Every criterion carries the six fields, plus two the composition problem forces:

- `active_when` — a predicate over the part's *configuration*, so "password-reset email arrives" exists only when `config.password_reset == true`. **No shipping format has this**; tag-based test selection (Cucumber tags) is the nearest ancestor and it gates suites, not criteria.
- `id` namespaced as `<part>/<instance>/<criterion>` — collision-proof by construction, following Arazzo's `$steps.*` namespacing and Salesforce's namespace-prefix lesson ([LWC guide](#)).

Worked example one — visual outcome (Auth part, login control):

```
part: auth@2
criteria:
  - id: auth/{instance}/AC-01
    active_when: always
    precondition:
      requires: [seeded_test_user]      # capability provided by assembly's
test tenancy
    trigger:
      surface: "{routes.login}"
      action: click
      control: { role: button, name: "Sign in" }
      after: { fill: { email: "{test_user.email}", password: "
{test_user.password}" } }
    observe:
      channel: http_response
      address: "POST {api_base}/auth/session"
      extract: { status: ".", body_field: "$.session.state" }
      decide:
```

```
- status == 200
- body_field == "active"
budget: { timeout_ms: 5000, retries: 2 }
evidence: [har_entry, trace_zip_action, response_body]
```

Worked example two — invisible outcome (Auth part, password-reset email):

```
- id: auth/{instance}/AC-04
  active_when: config.password_reset == true      # criterion exists only if
the feature does
  precondition:
    requires: [seeded_test_user, mail_capture]    # the part DECLARES the
channel it needs
  trigger:
    surface: "{routes.login}"
    action: click
    control: { role: link, name: "Forgot password" }
    after: { fill: { email: "{test_user.email}" }, submit: true }
  observe:
    channel: mail_capture                        # provided by assembly:
mailpit/mailosaur API
    address: "mailbox:{test_user.email}"
    extract: { subject: "$.subject", link: "$.html.links[0].href" }
    decide:
      - subject matches "(?i)reset"
      - link startswith "{app_base}/reset/"
    budget: { timeout_ms: 30000, poll_ms: 2000 }  # email is eventual; the
budget is the criterion
    evidence: [captured_message_json]
```

Verdict: no existing format suffices alone, but nothing here requires invention — only assembly. The six-field tuple is confirmed by the shipping formats' own required syntax; the two additions (`active_when` , capability-typed preconditions with declared observation channels) are the parts that exist nowhere and that your composition problem makes mandatory.

3. The composition rules

Derived from what the shipping formats do, and from the documented seam failures of platforms that got it wrong.

Rule 1 — Shared state travels as named capabilities with typed outputs, never as ambient assumptions. The Pact pattern (a state *name* as shared vocabulary between independently owned sides — `providerStates: [{name, params}]` ([provider docs](#))) merged with the Arazzo pattern (outputs consumed as `$steps.login.outputs.sessionToken`). Auth *provides* `authenticated_session`; Billing and Records *require* it. The checker topologically orders criteria by the provides/requires graph — exactly Playwright's `setup-project/ storageState` pattern ([auth docs](#)), lifted into declaration.

Rule 2 — Namespacing is structural, not conventional. Two lessons: Salesforce built mandatory namespace prefixes because flat naming breaks composition ([LWC guide](#)); the web-components world didn't, and its global `customElements` registry plus async upgrade produces documented havoc ([Carniato](#)). Neither Playwright nor Testing Library gives any test-id namespacing guidance — that silence is a finding. So: every part instance gets an id; every control's accessible name and test-id is prefixed with it; every criterion id is `part/instance/criterion`; two instances of the same part on one page are distinct by construction.

Rule 3 — Configuration gates criteria, and the gate is declared, not inferred. Each part's `config_schema` answers your question 12 (which provider, is reset on, who holds keys — the custodian fields from your gate's R6 belong here); `active_when` predicates make each criterion's existence a function of configuration. The checker resolves configuration *first*, then materialises the active criterion set — a criterion for a disabled feature is not skipped, it *does not exist*, which keeps pass-counts honest.

Rule 4 — Ordering comes from the dependency graph plus the trace, never from timestamps. `dependsOn` at the workflow level (Arazzo), provides/requires at the state level, and — where cross-part causality must be proven (Flow picked up what Forms submitted) — span-hierarchy assertions, which is the one thing Tracetest ships that nothing else has ([span-order](#)).

Rule 5 — Each part's envelope is declared, because the documented seam failures are envelope failures. The failure catalogue from the census: naming/registry collision, shared-dependency contention (WordPress's official troubleshooting doc institutionalises deactivate-all-and-bisect for "two or more plugins trying to use the same resources" ([docs](#))), substrate mutation (Shopify moved to sandboxed theme app extensions explicitly because injected code broke themes ([shopify.dev](#))), upgrade version skew (Mendix's

10 → 11 guide: React 19 removes `findDOMNode` so "all widgets require updating" ([docs](#))), and resource-envelope overflow (Bubble's 300s/50k limits mean two individually working blocks fail when composed at scale). The corresponding rule: a part declares its version-pinned dependencies, its resource envelope, and that it never mutates shared substrate; the assembly checker verifies the declarations don't conflict *before* the browser opens.

Verdict: every composition rule above is a generalisation of something one shipping system already does — and every seam failure in the catalogue is a system that skipped one of them. The rules are not speculative; the *combination* is.

4. What is unsolved — bluntly

1. **Nobody has shipped criteria that travel with a component and re-verify it inside the assembled application.** The near-misses each miss by exactly one dimension: Storybook play functions plus portable stories are executable criteria that physically travel with the component and run "in a live browser" via the test-runner ([docs](#)) — but they validate the component *in isolation*, not the app's wiring around it. Salesforce UTAM ships declarative JSON page objects with the platform's components — but they contain locators and interaction methods with *zero assertions* ([utam.dev](#)). Angular's component harnesses: same shape, "the assertions remain the consumer's job." Arazzo is mechanically evaluable and composable — but API-only, and its runner ecosystem is admitted to be "still forming" ([APIScout](#)). Web Platform Tests attach executable criteria to *specs*, not components. No documented reason was found for the gap — no post-mortem of an attempt; the evidence reads as *nobody has tried the full combination*, not as *tried and failed*.
2. **Config-conditional criteria exist nowhere.** No shipping format can say "this criterion exists iff this configuration is set." Tag-based suite selection is the nearest thing and it is the wrong granularity. `active_when` is a proposal in this report, not a citation.
3. **The invisible-outcome channel has no standard.** Interposed infrastructure per channel (mail catcher, webhook receiver, DB poller) driven by glue, or Tracetest's

spans with instrumentation coupling. Making channels a declared part-requirement (§2.3) is this report's answer; nobody's shipped it.

4. **Test-id and accessible-name namespacing has no authority to cite.** Playwright and Testing Library are silent; the proposal here (instance-prefixing) follows Salesforce's precedent but will be your convention, not an industry one.
5. **Webhook ordering is not assertable** at the receiver — Stripe says so of its own events. Criteria over webhook-driven flows must assert *sets and eventual states*, never sequences.
6. **Two census gaps:** Bubble and Mendix marketplace catalogues (and install counts) are JS-rendered and could not be enumerated — the parts table leans on the other twelve sources; and no primary-source Gartner/Forrester low-code figure survived verification, so none appears.

If someone has already built this: they have not. The verifiably closest assemblages are Storybook's play-function ecosystem (criteria travel, isolation only), UTAM (declarative, no assertions), and Arazzo with Redocly's runner (mechanical and composable, API-only, young). The combination — behavioural parts, six-field mechanical criteria, declared channels, config gating, instance namespacing, verified in the assembled app in a real browser — is unoccupied. Given your existing gate and the census in §1, the missing pieces are the two things labelled *proposal* above, and both are small.

Sources

Parts census: [Retool components](#) · [OutSystems Forge](#) · [Appsmith templates](#) · [Budibase templates](#) · [NocoBase handbook](#) · [Softr](#) · [ShipFast](#) · [SaaS Pegasus](#) · [Jetstream](#) · [Vercel SaaS starter](#) · [Wasp](#) / [Open SaaS](#) · [create-t3-app](#) · [Supabase](#) · [WordPress popular plugins](#) · [Bubble hard limits](#) · [Bubble reliability post-mortem](#) · [Mendix 10 → 11](#) · [OutSystems evaluation guide](#) / [custom code](#) · [WordPress troubleshooting](#) · [Shopify theme app extensions](#) · [Salesforce LWC namespaces](#) · [Carniato on web components](#) · [Stripe newsroom](#) (vendor) · [TechCrunch](#), [Stripe/Lemon Squeezy](#) · [Fortune](#), [Okta/Auth0](#) · [Novu](#) · [Cal.com](#). Formats: [Arazzo v1.1.0](#) · [Redocly on Arazzo](#) · [APIScout](#) · [Cucumber step definitions](#) · [Gherkin reference](#) · [Playwright assertions](#) / [actionability](#) / [auth](#) / [locators](#) · [Karate keywords](#) / [polling](#) / [DB](#)

· [StepCI](#) · [Hurl](#) · [Maestro](#) · [Tracetest](#) / [span order](#) · [Schemathesis](#) · [Microcks](#) · [Pact](#) / [pact-js](#) / [provider states](#) · [Gauge](#) · [Robot Framework](#) · [WebDriver BiDi](#) · [Mailosaur](#) · [Mailpit](#) · [Stripe webhooks](#) · [Awaitility](#) · [Storybook play functions](#) / [test-runner](#) / [portable stories](#) · [WPT](#) · [Custom Elements Manifest](#) · [Angular harnesses](#) · [UTAM](#) · [Testing Library](#) · [Open Feature](#) · [Curtis](#), [Component Contracts](#) (proposal).